

Materi Pembelajaran Pertemuan 2: Struktur Data Linear (Stack)

Latar Belakang

Pertemuan 2 bertujuan memperdalam pemahaman mahasiswa tentang struktur data linear dengan fokus pada **stack**, melanjutkan pengenalan array dari Pertemuan 1. Stack adalah struktur data yang mengikuti prinsip **LIFO (Last In, First Out)**, penting untuk aplikasi seperti **undo operation**, **evaluasi ekspresi matematis**, **manajemen panggilan fungsi**, dan **backtracking**. Materi ini mendukung pokok bahasan MK: **Macam-macam tipe data dan struktur data** (array, stack, dll.) dan **pengantar algoritma tingkat menengah**, dengan implementasi praktis menggunakan **Python** untuk kode dan **JavaScript (p5.js)** untuk animasi simulasi sederhana guna memperjelas konsep LIFO.

Tujuan Pembelajaran

- Mahasiswa memahami definisi, karakteristik, dan prinsip LIFO dari stack.
- Mahasiswa mampu mengimplementasikan operasi stack (**push**, **pop**, **is_empty**, **peek**) menggunakan Python.
- Mahasiswa dapat mengenali kegunaan stack dalam pemrograman dan menganalisis kelebihan serta kekurangannya.
- Mahasiswa dapat memvisualisasikan operasi stack melalui animasi sederhana menggunakan JavaScript.

Detail Pembahasan

1. Pengertian Stack

- **Definisi:** Stack adalah struktur data linear yang mengikuti prinsip **LIFO (Last In, First Out)**, di mana elemen yang terakhir masuk adalah yang pertama keluar.
- **Analogi:**
 - Tumpukan piring: Piring yang diletakkan terakhir diambil pertama.
 - Tumpukan buku: Buku teratas diangkat sebelum buku di bawahnya.
- **Karakteristik:**
 - **Operasi utama:**
 - **Push:** Menambah elemen ke ujung atas (top).

- **Pop:** Mengambil dan menghapus elemen teratas.
- **Is_empty:** Memeriksa apakah stack kosong.
- **Peek:** Mengembalikan elemen teratas tanpa menghapusnya.
- Struktur linear dengan akses terbatas hanya pada **top**.
- Elemen disimpan secara berurutan, baik dalam memori statis (array) maupun dinamis (list di Python).
- **Visualisasi:** Diagram stack sebagai tumpukan kotak, dengan panah menunjukkan top.

2. Operasi Stack

- **Push:**
 - Menambahkan elemen baru ke posisi top.
 - Contoh: Push “Buku1” ke stack kosong → stack berisi [“Buku1”].
- **Pop:**
 - Mengambil elemen top dan menghapusnya dari stack.
 - Contoh: Pop dari [“Buku1”, “Buku2”] → mengembalikan “Buku2”, stack menjadi [“Buku1”].
- **Is_empty:**
 - Mengembalikan True jika stack kosong, False jika tidak.
 - Contoh: Stack [] → True; Stack [“Buku1”] → False.
- **Peek:**
 - Melihat elemen top tanpa menghapusnya.
 - Contoh: Peek pada [“Buku1”, “Buku2”] → mengembalikan “Buku2”.
- **Implementasi di Python:**
 - Menggunakan `list` untuk kesederhanaan.
 - `append()` untuk push, `pop()` untuk pop.
 - Penanganan kasus khusus: Memeriksa kekosongan sebelum pop untuk mencegah error.
- **Catatan:** Operasi bersifat **konstan** (O(1)) untuk push dan pop, karena hanya mengakses top.

3. Kegunaan Stack

- **Undo Operation:**
 - Menyimpan riwayat aksi (misalnya, teks yang diketik di editor) untuk dibatalkan.
 - Contoh: Mengetik “A”, “B”, “C” → pop “C” untuk undo.
- **Evaluasi Ekspresi Matematis:**
 - Konversi infix ke postfix (misalnya, `3 + 4 * 2` menjadi `3 4 2 * +`).
 - Mengevaluasi ekspresi postfix menggunakan stack.

- **Manajemen Panggilan Fungsi:**
 - Call stack menyimpan informasi fungsi yang dipanggil (alamat kembali, parameter).
 - Contoh: Fungsi rekursif seperti faktorial.
- **Backtracking:**
 - Menyimpan langkah sementara dalam algoritma seperti maze solving atau N-Queens.
 - Contoh: Menyimpan jalur percobaan, kembali ke langkah sebelumnya jika gagal.
- **Lainnya:**
 - Parsing sintaks (misalnya, cek tanda kurung seimbang: `{[()]}`).
 - Simulasi rekursi secara iteratif.

4. Implementasi Stack di Python

- **Pendekatan:**
 - Menggunakan `class Stack` untuk enkapsulasi.
 - List sebagai penyimpanan internal untuk fleksibilitas.
- **Contoh Kode:**

```
class Stack:
    def __init__(self):
        self.items = []

    def push(self, item):
        self.items.append(item)

    def pop(self):
        if not self.is_empty():
            return self.items.pop()
        return None

    def peek(self):
        if not self.is_empty():
            return self.items[-1]
        return None

    def is_empty(self):
        return len(self.items) == 0
```

```

def size(self):
    return len(self.items)

# Pengujian
stack = Stack()
stack.push("Buku1")
stack.push("Buku2")
print("Peek:", stack.peek()) # Output: Buku2
print("Pop:", stack.pop())   # Output: Buku2
print("Is empty?", stack.is_empty()) # Output: False
print("Size:", stack.size()) # Output: 1

```

- **Penanganan Kasus Khusus:**
 - Pop pada stack kosong: Mengembalikan **None** atau pesan error.
 - Peek pada stack kosong: Mengembalikan **None**.
- **Fleksibilitas:**
 - Dapat menyimpan tipe data apa pun (string, int, dll.).
 - Mudah dimodifikasi untuk batasan ukuran jika diperlukan.

5. Visualisasi Stack dengan JavaScript (p5.js)

- **Tujuan:** Memvisualisasikan operasi push dan pop untuk memperjelas LIFO.
- **Pendekatan:**
 - Menggunakan **p5.js** untuk menggambar tumpukan kotak secara interaktif.
 - Setiap kotak mewakili elemen stack, dengan animasi saat push/pop.
- **Contoh Kode JavaScript:**

```

let stack = [];
let action = "idle";

function setup() {
  createCanvas(400, 400);
  textAlign(CENTER, CENTER);
}

function draw() {
  background(220);
  // Gambar stack

```

```

for (let i = 0; i < stack.length; i++) {
    fill(100, 150, 255);
    rect(150, 300 - i * 50, 100, 40);
    fill(0);
    text(stack[i], 200, 320 - i * 50);
}
// Label top
if (stack.length > 0) {
    text("Top", 120, 320 - (stack.length - 1) * 50);
}
}

function mousePressed() {
    if (mouseX > 50 && mouseX < 150 && mouseY > 350) {
        // Tombol Push
        stack.push("Item" + (stack.length + 1));
    } else if (mouseX > 250 && mouseX < 350 && mouseY > 350) {
        // Tombol Pop
        stack.pop();
    }
}
}

```

- **Penjelasan:**

- Kanvas menampilkan stack sebagai kotak bertumpuk.
- Klik tombol “Push” (area kiri) menambah elemen.
- Klik tombol “Pop” (area kanan) menghapus elemen teratas.
- Label “Top” menunjukkan elemen teratas.

- **Output:** Animasi interaktif di browser, menunjukkan elemen masuk/keluar.

6. Analisis Sederhana

- **Kelebihan:**

- Implementasi sederhana, cocok untuk operasi LIFO.
- Efisiensi tinggi: $O(1)$ untuk push, pop, peek, is_empty.
- Fleksibel di Python karena list bersifat dinamis.

- **Kekurangan:**

- Akses terbatas hanya pada top (tidak seperti array).

- Jika menggunakan array statis (di bahasa lain), ukuran terbatas.
- Pencarian elemen tertentu memerlukan $O(n)$.

- **Kompleksitas:**

Operasi	Kompleksitas Waktu
Push	$O(1)$
Pop	$O(1)$
Peek	$O(1)$
Is_empty	$O(1)$
Search	$O(n)$

- **Perbandingan:**

- Dibandingkan array: Stack lebih terbatas tetapi lebih terfokus.
- Dibandingkan queue: Stack untuk LIFO, queue untuk FIFO.

Outline Presentasi (PPT)

Berikut adalah struktur slide presentasi untuk kuliah 50 menit, dirancang interaktif dan visual:

1. Slide 1: Judul dan Pengenalan

- Judul: “Pertemuan 2: Struktur Data Linear - Stack”
- Tanggal: 24 Maret 2025
- Tujuan:
 - Memahami stack dan prinsip LIFO
 - Mengimplementasikan stack di Python
 - Memvisualisasikan operasi stack
- Konten: Selamat datang, kaitan dengan Pertemuan 1 (array)

2. Slide 2: Definisi dan Analogi Stack

- Definisi: Struktur data LIFO
- Analogi: Tumpukan piring atau buku
- Diagram: Kotak bertumpuk dengan label top
- Pertanyaan: Apa contoh LIFO di kehidupan sehari-hari?

3. Slide 3: Operasi Stack

- Operasi: Push, Pop, Is_empty, Peek
- Ilustrasi:
 - Push: Tambah kotak ke tumpukan
 - Pop: Ambil kotak teratas
- Tabel:

Operasi	Fungsi
Push	Tambah elemen ke top
Pop	Hapus dan kembalikan top
Is_empty	Cek kekosongan
Peek	Lihat top tanpa hapus

4. Slide 4: Kegunaan Stack

- Daftar aplikasi:
 - Undo operation
 - Evaluasi ekspresi (contoh: $(3+4)*2$)
 - Call stack
 - Backtracking
- Visual: Gambar undo di editor teks
- Diskusi: Kapan stack lebih baik dari list biasa?

5. Slide 5: Implementasi di Python

- Kode:

```
class Stack:
    def __init__(self):
        self.items = []
    def push(self, item):
        self.items.append(item)
    def pop(self):
        if not self.is_empty():
            return self.items.pop()
        return None
    def is_empty(self):
        return len(self.items) == 0
```

- Demo: Push “Item1”, “Item2”, pop, cek is_empty
- Output contoh: “Pop: Item2”, “Is empty? False”

6. Slide 6: Visualisasi dengan JavaScript

- Penjelasan: Animasi push/pop menggunakan p5.js
- Gambar: Screenshot animasi tumpukan kotak
- Instruksi: Akses demo di browser (opsional untuk mahasiswa)
- Tautan: (Jika disediakan online)

7. Slide 7: Analisis Stack

- Kelebihan: $O(1)$ untuk operasi utrif
- Kekurangan: Akses terbatas, ukuran statis (non-Python)
- Tabel kompleksitas (lihat tabel di atas)
- Perbandingan: Stack vs Array vs Queue
- Pertanyaan: Mengapa stack cocok untuk rekursi?

8. Slide 8: Kesimpulan dan Tugas

- Ringkasan: Stack = LIFO, efisien, banyak aplikasi
- Tugas: Lihat bagian Penugasan
- Persiapan Pertemuan 3: Strategi algoritma dan diagram alir
- Penutup: Pertanyaan dari mahasiswa

Aktivitas Pembelajaran (100 Menit)

• Kuliah (50 menit):

- Penyampaian slide dengan pendekatan interaktif:
 - Diskusi singkat (5 menit): Mahasiswa menyebutkan contoh LIFO.
 - Demonstrasi kode Python langsung di IDE (10 menit).
 - Penjelasan animasi JavaScript via screenshot atau demo singkat (5 menit).
- Penekanan pada analogi dan aplikasi untuk menarik minat.

• Praktik (50 menit):

- **Latihan 1 (20 menit):** Mahasiswa menulis kode stack:
 - Buat stack, push 4 elemen (misalnya, “A”, “B”, “C”, “D”).
 - Lakukan pop 2 kali, cetak hasil.
 - Cek is_empty dan size.
- **Latihan 2 (20 menit):** Modifikasi kode untuk menambahkan fungsi peek.

- **Diskusi Kelompok (10 menit):** Mahasiswa berpasangan, menjelaskan kode mereka kepada teman (mendukung CPMK 1.1).
- Instruktur memandu debugging (misalnya, error pop pada stack kosong).

Penugasan

1. Tugas Individu (Dikumpulkan sebelum Pertemuan 3):

- **Bagian 1: Kode Python**

- Buat program Python untuk stack dengan operasi:
 - Push 5 elemen (contoh: nama kota seperti “Jakarta”, “Bandung”, dll.).
 - Pop 3 elemen, cetak setiap elemen yang di-pop.
 - Gunakan peek untuk melihat elemen top setelah pop.
 - Cek `is_empty` dan `size` setelah semua operasi.
- Simpan kode dalam file `.py`.

- **Bagian 2: Analisis**

- Tulis penjelasan (150-200 kata):
 - Bagaimana stack dapat digunakan untuk memeriksa tanda kurung seimbang (misalnya, `{[()]}`)?
 - Apa kelebihan stack dibandingkan list biasa untuk kasus ini?
- Simpan dalam file `.docx` atau `.pdf`.

- **Kriteria Penilaian:**

- Kebenaran kode: 40%
- Kelengkapan fungsi (push, pop, peek, dll.): 20%
- Kualitas penjelasan: 30%
- Ketepatan waktu: 10%

- **Batas Waktu:** 1 hari sebelum Pertemuan 3, via LMS atau email.

2. Tugas Reflektif (Diskusi di Pertemuan 3):

- Tulis 1-2 kalimat tentang aplikasi stack lain (selain yang dibahas). Contoh: “Stack digunakan di browser untuk menyimpan riwayat navigasi.”
- Dibahas singkat di awal Pertemuan 3 untuk mendorong CPMK 1.1.

Referensi

- **Pustaka Utama:**

- Thomas H. Cormen et al., *Introduction to Algorithms*, Third Edition, 2014 (Bab Stack).
- Adam Drozdek, *Data Structures and Algorithms in C++*, 2012 (Bab Stack, diadaptasi ke Python).
- Munir, R. dan Lidya, L., *Algoritma dan Pemrograman*, 2016 (Bab Struktur Data).
- **Pustaka Pendukung:**
 - Rosa, A.S., *Struktur Data*, 2018 (Bab Stack).
- **Sumber Tambahan:**
 - GeeksforGeeks: Stack Data Structure.
 - W3Schools: Python Lists.
 - p5.js Documentation (untuk animasi).

Catatan Tambahan

- **Asumsi:** Mahasiswa memiliki dasar pemrograman Python (variabel, list, fungsi), sesuai pokok bahasan MK tentang review pemrograman dasar.
- **Fasilitas:**
 - Lab komputer dengan Python 3.8+ (VS Code/Jupyter Notebook).
 - Browser untuk demo p5.js (opsional, jika animasi dijalankan langsung).
- **Tantangan Potensial:**
 - Mahasiswa baru dengan LIFO: Diperkuat dengan analogi dan visualisasi.
 - Error handling: Instruktur menjelaskan pentingnya `is_empty()`.
 - Waktu praktik: Jika terbatas, latihan peek dapat dijadikan tugas.
- **Kaitan JavaScript:** Animasi sederhana digunakan untuk visualisasi, sesuai rencana MK. Demo p5.js bersifat opsional untuk menghindari kelebihan beban di Pertemuan 2.
- **Progresi Pembelajaran:** Materi ini membangun fondasi untuk topik lanjutan seperti queue (Pertemuan 5) dan evaluasi ekspresi (kemudian).

Contoh Hasil Praktik

Kode yang diharapkan dari latihan:

```
class Stack:
    def __init__(self):
        self.items = []

    def push(self, item):
        self.items.append(item)
```

```
def pop(self):  
    if not self.is_empty():  
        return self.items.pop()  
    return None
```

```
def peek(self):  
    if not self.is_empty():  
        return self.items[-1]  
    return None
```

```
def is_empty(self):  
    return len(self.items) == 0
```

```
def size(self):  
    return len(self.items)
```

Pengujian

```
stack = Stack()  
stack.push("A")  
stack.push("B")  
stack.push("C")  
stack.push("D")  
print("Pop:", stack.pop()) # Output: D  
print("Pop:", stack.pop()) # Output: C  
print("Peek:", stack.peek()) # Output: B  
print("Size:", stack.size()) # Output: 2  
print("Is empty?", stack.is_empty()) # Output: False
```